



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Dominik Rybski
Nr albumu studenta: 71355

"Programowanie Obiektowe – System obsługi parkingu"

Prowadzący: mgr inż. Ewa Żesławska

Projekt

Rzeszów 2025/2026

Spis treści

Wstęp	4
1 Opis założeń i cele projektu	4
1.1 Założenia projektu	4
1.2 Cele projektu	4
1.3 Wymagania funkcjonalne	4
1.4 Wymagania niefunkcjonalne	4
2 Opis struktury projektu	5
2.1 Diagram klas i opis struktury	5
2.2 Opis techniczny i narzędzia	5
2.3 Zarządzanie danymi oraz baza danych	6
3 Harmonogram realizacji projektu	7
3.1 Diagram Ganta	7
3.2 Opis etapów realizacji	7
4 Repozytorium i system kontroli wersji	8
5 Prezentacja warstwy użytkowej projektu	9
5.1 Opis aplikacji	9
5.2 Prezentacja funkcjonalności	10
6 Podsumowanie	11
6.1 Zrealizowane prace	11
6.2 Dalsze prace rozwojowe	11
7 Literatura	12
Spis rysunków	13
Streszczenie	14

Rozdział 1

Opis założeń i cele projektu

1.1 Założenia projektu

Głównym założeniem projektu jest stworzenie nowoczesnego, zautomatyzowanego systemu zarządzania przestrzenią parkingową. Aplikacja symuluje rzeczywiste warunki eksploatacji parkingu na dwuwymiarowej siatce o wymiarach 6 x 5. System uwzględnia specyficzną infrastrukturę drogową (drogi przejazdowe) oraz zróżnicowane gabaryty pojazdów, zapewniając trwałość danych dzięki integracji z relacyjną bazą danych SQL Server.

1.2 Cele projektu

- **Cel techniczny:** Implementacja stabilnej aplikacji konsolowej w środowisku .NET 9, która zarządza stanem parkingu w pamięci operacyjnej i synchronizuje go z bazą danych.
- **Cel edukacyjny:** Praktyczne zastosowanie paradygmatów programowania obiektowego (OOP), w tym dziedziczenia klas, polimorfizmu metod oraz hermetyzacji danych.
- **Cel użytkowy:** Stworzenie intuicyjnego interfejsu CLI (Command Line Interface) umożliwiającego szybką ewidencję przyjazdów i odjazdów.

1.3 Wymagania funkcjonalne

- **Obsługa menu:** Interaktywne menu umożliwiające wybór operacji (1-5).
- **Zarządzanie pojazdami:** Dodawanie i usuwanie pojazdów czterech typów (Motocykl, Samochód, Ciężarówka, Autobus) z walidacją miejsca.
- **Wizualizacja:** Graficzna reprezentacja siatki parkingu z oznaczeniem miejsc zajętych i wolnych.
- **Persystencja:** Rejestracja każdej transakcji w bazie danych SQL Server i możliwość podglądu historii.

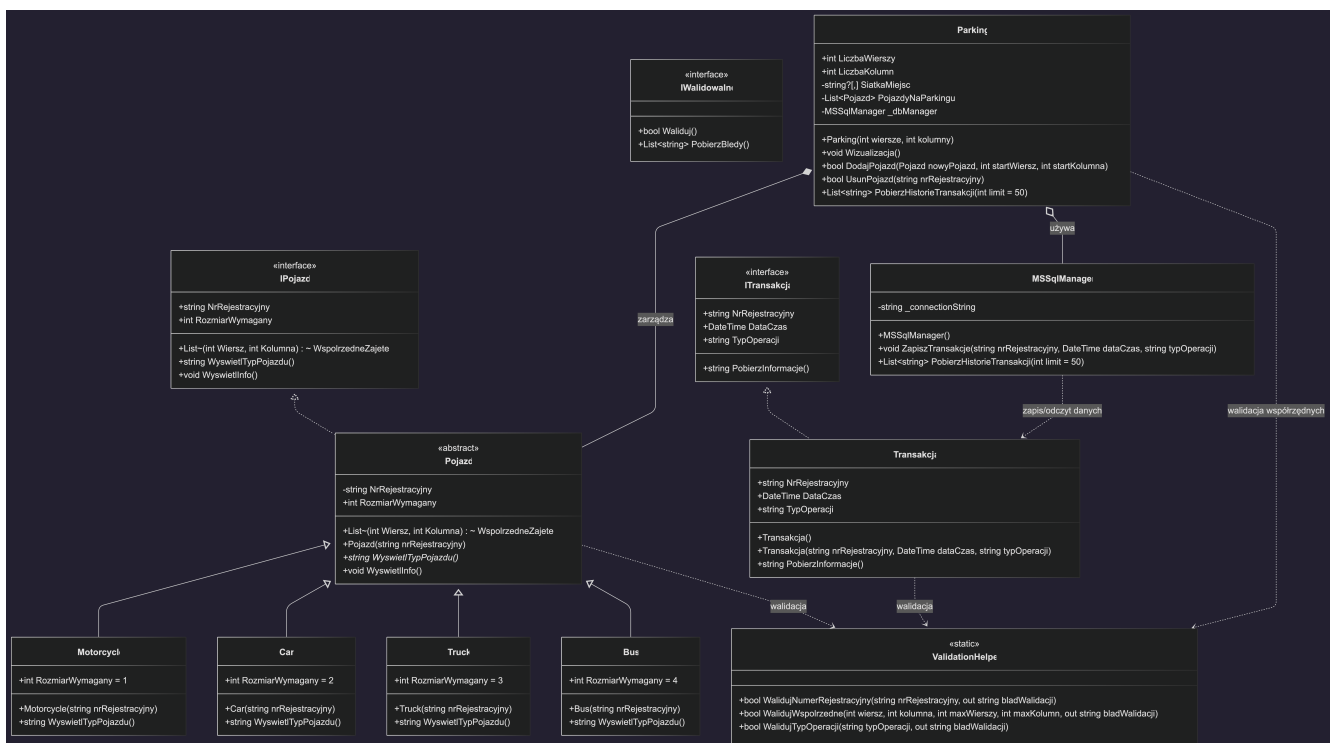
1.4 Wymagania niefunkcjonalne

- **Bezpieczeństwo:** Ochrona przed SQL Injection poprzez parametryzację zapytań.
- **Modularność:** Podział kodu na warstwy (Models, Services, Helpers, Interfaces).
- **Konfiguracja:** Przechowywanie parametrów połączenia w pliku `appsettings.json`.

Rozdział 2

Opis struktury projektu

2.1 Diagram klas i opis struktury



Rysunek 2.1: Diagram Klas

System został zaprojektowany w oparciu o czystą hierarchię klas. Fundamentem jest klasa abstrakcyjna *Pojazd*, która definiuje cechy wspólne dla każdego obiektu (nr rejestracyjny, lista współrzędnych). Klasy konkretne: *Car*, *Motorcycle*, *Truck* oraz *Bus* implementują specyficzny dla danego typu rozmiar wymagany na siatce.

2.2 Opis techniczny i narzędzia

- **Język:** C# na platformie .NET 9.0.
- **Narzędzia:** Visual Studio Code na systemie macOS.
- **Minimalne wymagania:** Środowisko uruchomieniowe .NET 9, dostęp do bazy SQL Server.

2.3 Zarządzanie danymi oraz baza danych

☐		 Id int			* NrRejestracyjny nvarchar(50)			* DataCzas datetime			* TypOperacji nvarchar(50)		
☐	>	1			WA987			2026-02-10 11:05:4			Przyjazd		
☐	>	2			KR123			2026-02-10 11:05:4			Przyjazd		
☐	>	3			GD000			2026-02-10 11:05:4			Przyjazd		
☐	>	4			WA987			2026-02-10 11:05:4			Odjazd		
☐	>	5			KR123			2026-02-10 11:05:4			Odjazd		
☐	>	6			GD000			2026-02-10 11:05:4			Odjazd		
☐	>	7			WA987			2026-02-10 11:29:2			Przyjazd		
☐	>	8			KR123			2026-02-10 11:29:2			Przyjazd		
☐	>	9			GD000			2026-02-10 11:29:2			Przyjazd		
☐	>	10			WA987			2026-02-10 11:29:2			Odjazd		

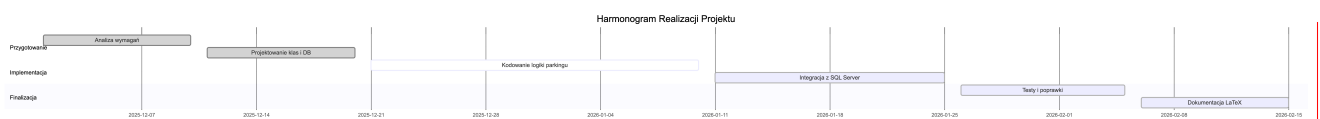
Rysunek 2.2: Wykaz bazy danych

Zarządzanie informacjami odbywa się dwutorowo. Bieżący stan siatki przechowywany jest w pamięci RAM klasy `Parking.cs`. Historia transakcji składowana jest trwale w bazie `ParkingDB` (tabela `Transakcje`). Do komunikacji wykorzystano technologię ADO.NET i bibliotekę `System.Data.SqlClient`.

Rozdział 3

Harmonogram realizacji projektu

3.1 Diagram Ganta



Rysunek 3.1: Diagram Ganta

3.2 Opis etapów realizacji

1. **Analiza (tydzień 1):** Opracowanie wymagań i logiki dróg przejazdowych ($row \pmod{3} = 2$).
2. **Projektowanie (tydzień 2):** Przygotowanie diagramu klas oraz schematu tabeli SQL.
3. **Kodowanie (tydzień 3-4):** Implementacja logiki alokacji miejsc oraz warstwy usług bazy danych.
4. **Testy (tydzień 5):** Weryfikacja scenariuszy brzegowych i finalizacja dokumentacji LaTeX.

Rozdział 4

Repozytorium i system kontroli wersji

Projekt jest zarządzany przy użyciu systemu kontroli wersji Git. Kod źródłowy jest hostowany w serwisie GitHub pod adresem:

`https://github.com/DominikRybski/ParkingSystem`

Rozdział 5

Prezentacja warstwy użytkowej projektu

5.1 Opis aplikacji

```
--- AKTUALNY STAN PARKINGU ---
[X][X][ ][ ][ ]
[X][X][ ][ ][ ]
===== PRZEJAZD
[ ][ ][ ][X][X]
[ ][ ][ ][X][X]
===== PRZEJAZD

--- MENU ---
1. Dodaj pojazd
2. Usuń pojazd
3. Pokaż wizualizację parkingu
4. Pokaż historię transakcji
5. Wyjście
Twój wybór: █
```

Rysunek 5.1: Wizualizacja interfejsu

Interfejs CLI zapewnia przejrzystą obsługę systemu. Użytkownik komunikuje się z aplikacją poprzez wpisywanie numerów opcji. System wizualizuje parking w formie macierzy, gdzie [] to wolne miejsce, a [X] to miejsce zajęte.

5.2 Prezentacja funkcjonalności

```
--- MENU ---
1. Dodaj pojazd
2. Usuń pojazd
3. Pokaż wizualizację parkingu
4. Pokaż historię transakcji
5. Wyjście
Twój wybór: 1

Typ pojazdu:
1. Motocykl (1 miejsce)
2. Samochód osobowy (2 miejsca)
3. Ciężarówka (3 miejsca)
4. Autobus (2x2)
Wybierz typ (1-4): 4
Podaj numer rejestracyjny: WWA1234
Podaj wiersz startowy: 2
Podaj kolumnę startową: 2
Błąd: Nie można parkować na przejeździe.
```

Rysunek 5.2: Wizualizacja walidacji

Na powyższym zrzucie ekranu widoczna jest siatka parkingu z poprawnie wygenerowanymi drogami przejazdowymi, na których parkowanie jest zablokowane przez system walidacji.

Rozdział 6

Podsumowanie

6.1 Zrealizowane prace

W ramach projektu zaimplementowano pełną logikę zarządzania parkingiem. System poprawnie obsługuje gabaryty pojazdów, waliduje numery rejestracyjne oraz współrzędne, a także zapewnia trwałość informacji poprzez logowanie transakcji do bazy danych.

6.2 Dalsze prace rozwojowe

W przyszłości planowana jest migracja interfejsu na technologię WPF (GUI) oraz dodanie modułu obliczania opłat za postój na podstawie różnicy czasu przyjazdu i odjazdu.

Rozdział 7

Literatura

1. Dokumentacja Microsoft .NET: <https://learn.microsoft.com/dotnet/>
2. Dokumentacja Microsoft SQL Server: <https://learn.microsoft.com/sql/>

Spis rysunków

2.1	Diagram Klas	5
2.2	Wykaz bazy danych	6
3.1	Diagram Ganta	7
5.1	Wizualizacja interfejsu	9
5.2	Wizualizacja walidacji	10

Streszczenie projektu

Programowanie Obiektowe – System obsługi parkingu

Autor: Dominik Rybski

Promotor: mgr inż. Ewa Żesławska

Słowa kluczowe: System obsługi parkingu

Parking jest prostokątem zawierającym miejsca w ustalonej liczbie wierszy i kolumn. Przejazdy są co drugi wiersz. System pozwala na wprowadzanie nowych pojazdów (przyjazd), wyświetlenie informacji o wszystkich, oraz na usuwanie ich z listy (odjazd). System obsługuje każdy typ pojazdu. Może nim być: • motocykl – zajmuje 1 miejsce • samochód osobowy – zajmuje 2 miejsca obok siebie. • autobus – zajmuje 4 miejsca (w dwóch sąsiednich wierszach i kolumnach) • W klasie Parking napisz metodę wykonującą wizualizację parkingu (w konsoli znakami tekstowymi). Pojazd musi posiadać informacje o współrzędnych pól na których się znajduje, a także nr rejestracyjny. Można dodać nowy pojazd tylko wtedy, gdy jest puste miejsce na podanych współrzędnych. Dodatkowo każdy pojazd musi wyświetlić informacje o sobie (nr rejestracyjny i pola zajmowane, oraz czym on jest – samochód, motor, autobus). Dodatkowo każdy przyjazd i odjazd jest zapisywany w liście opisującej stan parkingu. W ramach listy – można wyszukać pojazd (wg np. rejestracyjnego lub daty i godziny przyjazdu/odjazdu).

The University of Information Technology and Management in Rzeszow
Faculty of Applied Information Technology

Thesis Summary

Object-Oriented Programming – Parking Management System

Author: Dominik Rybski

Supervisor: mgr inż. Ewa Żesławska

Key words: Parking Management System

The Parking Management System must simulate a rectangular grid with designated aisles every second row. It will handle CRUD operations (Create, Read, Update, Delete) for vehicles, including Motorcycles (1 spot), Passenger Cars (2 adjacent spots), and Buses (4 contiguous spots). The system requires a console method for visualization and must maintain a transaction log of all arrivals and departures, allowing data retrieval based on vehicle registration or timestamp. Vehicles must store their unique registration number and occupied coordinate array.